

سلام. امیدوارم حالتون خوب باشه. اینو می‌نویسم برای گزارش فاز ۲ پروژه‌ی رمزنگاری. توی این بخش از ما خواسته شده بود که بیایم و یه ساختار خیلی ساده از PKI درست کنیم ولی قبلش یه سری پیاده‌سازی‌ها رو از مثلا RSA و DSA انجام بدیم. در ادامه همه‌ی این‌ها رو بخش به بخش توضیح می‌دم. نکته‌ای که هست اینه که این کد ۵۵۰ خطه و خب قطعا نمی‌شه همشو توضیح داد چون هم متن زیاد می‌شه هم حوصله‌ی شما سر می‌ره. من فقط نکته‌های مهم کد رو توضیح می‌دم و توضیح بخش‌های اضافه‌تر داخل تحویل داده خواهد شد.

بخش اول) تولید عدد اول بزرگ و تست اون با استفاده از Miller-Rabin: تو این قسمت اول باید یه عدد بزرگ درست می‌کردیم و بعد باید عددی که تولید کردیم رو با این تابع تست می‌کردیم و می‌دیدیم که آیا عدد اول هست یا نه. کد این بخش رو من این پایین می‌ذارم:

```

1 def is_prime_miller_rabin(n: int, k: int):
2     """
3     Perform the Miller-Rabin primality test on a number.
4
5     :param n: The number to test for primality.
6     :param k: The number of iterations of the test to perform.
7     :return: True if n is probably prime, False if n is composite.
8     """
9     # Edge cases
10    if n <= 1:
11        return False
12    if n <= 3:
13        return True
14    if n % 2 == 0:
15        return False
16
17    # Write n - 1 as 2^r * d where d is odd
18    r, d = 0, n - 1
19    while d % 2 == 0:
20        r += 1
21        d //= 2
22
23    # Perform k iterations of the test
24    for _ in range(k):
25        a = random.randint(2, n - 2)
26        x = pow(a, d, n) # Compute a^d % n
27        if x == 1 or x == n - 1:
28            continue
29
30        for _ in range(r - 1):
31            x = pow(x, 2, n)
32            if x == n - 1:
33                break
34        else:
35            return False
36
37    return True
38

```

```

1 def generate_large_prime(bits: int, k: int):
2     """
3     Generate a large prime number with the specified bit length.
4
5     :param bits: Number of bits for the prime number.
6     :param k: Number of iterations for the Miller-Rabin test.
7     :return: A large prime number.
8     """
9     while True:
10         candidate = random.getrandbits(bits)
11         candidate |= (1 << (bits - 1)) | 1 # Ensure it's odd and has the required bit length
12         if is_prime_miller_rabin(candidate, k):
13             return candidate
14

```

بخش دوم) ساختن Public & Private key مشتری با استفاده از RSA: تو این قسمت باید می‌اومدیم و کلیدهامون رو با استفاده از RSA درست می‌کردیم. بعد این‌ها رو داخل یه فایل pem ذخیره می‌کردیم. چیز خاصی کلا نداره و من کد این بخش رو می‌ذارم این پایین:

```

1 def generate_rsa_key_pair():
2     """
3     Generate RSA keys and save them in proper PEM format.
4     """
5     print("Generating RSA Key Pair...")
6     bits = 2048 # For real applications, this should be at least 2048 or higher.
7     k = 40 # Number of iterations for primality test
8     p = generate_large_prime(bits // 2, k)
9     q = generate_large_prime(bits // 2, k)
10    n = p * q
11    phi = (p - 1) * (q - 1)
12
13    # Select a random e such that gcd(e, phi) = 1
14    while True:
15        e = random.randint(2, phi - 1)
16        if gcd(e, phi) == 1:
17            break
18
19    # Calculate the private key d
20    d = mod_inverse(e, phi)
21
22    # Create the keys
23    private_key = {"p": p, "q": q, "n": n, "d": d, "e": e}
24    public_key = {"n": n, "e": e}
25
26    # Encode the keys to Base64
27    private_key_encoded = base64.b64encode(json.dumps(private_key).encode()).decode()
28    public_key_encoded = base64.b64encode(json.dumps(public_key).encode()).decode()
29
30    # Save the keys in PEM format
31    os.makedirs(CLIENT_DIR, exist_ok=True)
32    with open(os.path.join(CLIENT_DIR, "client_private_key.pem"), "w") as f:
33        f.write(f"-----BEGIN RSA PRIVATE KEY-----\n{private_key_encoded}\n\n-----END RSA PRIVATE KEY-----")
34
35    with open(os.path.join(CLIENT_DIR, "client_public_key.pem"), "w") as f:
36        f.write(f"-----BEGIN RSA PUBLIC KEY-----\n{public_key_encoded}\n\n-----END RSA PUBLIC KEY-----")
37
38    print("RSA Key Pair Generated and Saved in PEM Format.")
39    return private_key, public_key
40

```

این هم شکلی از فرمت فایل pem هست:

بخش سوم) تولید Public & Private key سیای سرور برای امضا و تایید: توی این قسمت هم باید کلیدهای عمومی و خصوصی سیای سرور رو درست می‌کردیم با استفاده از امضای DSA. تنها نکته‌ای که کد من داره برای این بخش اینه که من خودم مقدار q رو کوچیک فرض کردم که بعدا سر محاسباتش خیلی طول نکشه. کلا هم باید q رو یه جوری انتخاب کنیم که $p-1$ رو بشمره وگرنه کلا امضا غلط می‌شه. کد این بخش هم این پایین گذاشتم:

```
1 def generate_dsa_key_pair():
2     """
3     Generate DSA keys using manual prime generation and parameters.
4     Save the private key and public key in proper PEM format.
5     """
6     print("Generating DSA Key Pair...")
7     # First, generate q
8     q = generate_large_prime(20, 40)
9
10    # Then find p so that  $p = k * q + 1$  is prime
11    while True:
12        k = random.randint(2, (1 << 8)) # For demo, pick a small search space
13        p_candidate = k * q + 1
14        if is_prime_miller_rabin(p_candidate, 40):
15            p = p_candidate
16            break
17
18    h = random.randint(2, p - 2)
19    # Compute g modulo p, not q
20    g = pow(h, (p - 1) // q, p)
21
22    x = random.randint(1, q - 1) # Private key x
23    y = pow(g, x, p)             # Public key  $y = g^x \text{ mod } p$ 
24
25    # Prepare the keys in dictionary format
26    private_key = {"p": p, "q": q, "g": g, "x": x}
27    public_key = {"p": p, "q": q, "g": g, "y": y}
28
29    # Encode the keys to Base64
30    private_key_encoded = base64.b64encode(json.dumps(private_key).encode()).decode()
31    public_key_encoded = base64.b64encode(json.dumps(public_key).encode()).decode()
32
33    # Save the keys in PEM format
34    os.makedirs(CA_DIR, exist_ok=True)
35    with open(os.path.join(CA_DIR, "ca_private_key.pem"), "w") as f:
36        f.write(f"-----BEGIN DSA PRIVATE KEY-----\n{private_key_encoded}\n\n\n\nEND DSA PRIVATE KEY-----")
37
38    with open(os.path.join(CA_DIR, "ca_public_key.pem"), "w") as f:
39        f.write(f"-----BEGIN DSA PUBLIC KEY-----\n{public_key_encoded}\n\n\n\nEND DSA PUBLIC KEY-----")
40
41    print("DSA Key Pair Generated and Saved in PEM Format.")
42    return private_key, public_key
43
```

بخش چهارم) درخواست امضای کلید عمومی مشتری و امضای اون توسط سی‌ای سرور: توی
این بخش مشتری یه فایل json درست می‌کنه و داخل اون کلید عمومی خودش (که با RSA تولید شده) رو قرار می‌ده با یه سری اطلاعات دیگه و این فایل رو قرار می‌ده داخل یه دایرکتوری مشترک با سی‌ای سرور. بعد سی‌ای سرور اون درخواست رو برمی‌داره و امضا می‌کنه و قرارش می‌ده داخل یه دایرکتوری دیگه (من قرار دادم داخل یه دایرکتوری دیگه. می‌تونه توی همون دایرکتوری قبلی هم باشه.) و به این ترتیب الان مشتری یه سرتیفیکیت داره که می‌تونه به بقیه بده اون رو. کد این بخش رو می‌ذارم این پایین:

```
1 def create_csr():
2     print("Client: Creating CSR...")
3     csr = {
4         "common_name": "client1",
5         "organization": "ExampleCorp",
6         "organization_unit": "IT",
7         "country": "US",
8         "state": "California",
9         "locality": "San Francisco",
10        "email": "client1@example.com",
11    }
12
13    # Load the public key from the client's RSA public key file
14    with open(os.path.join(CLIENT_DIR, "client_public_key.pem"), "r") as f:
15        public_key_pem = f.read()
16        csr["public_key"] = public_key_pem # Include public key as PEM format
17
18    # Save the CSR to the shared directory
19    os.makedirs(SHARED_DIR_CSR, exist_ok=True)
20    with open(os.path.join(SHARED_DIR_CSR, "csr.json"), "w") as f:
21        json.dump(csr, f, indent=4)
22
23    print("Client: CSR created and saved to shared directory.")
24
```

```

1 def sign_csr_and_generate_cert():
2     """
3     CA signs the CSR and generates the certificate using the updated PEM structure.
4     """
5     print("CA: Signing CSR and generating certificate...")
6
7     # Load the CA private key from the PEM file
8     with open(os.path.join(CA_DIR, "ca_private_key.pem"), "r") as f:
9         private_key_pem = f.read().splitlines()[1] # Extract the Base64 content
10        ca_private_key = json.loads(base64.b64decode(private_key_pem)) # Decode and Load JSON
11        x = ca_private_key["x"]
12
13    # Load the CA public key from the PEM file
14    with open(os.path.join(CA_DIR, "ca_public_key.pem"), "r") as f:
15        public_key_pem = f.read().splitlines()[1] # Extract the Base64 content
16        ca_public_key = json.loads(base64.b64decode(public_key_pem)) # Decode and Load JSON
17        y = ca_public_key["y"]
18        p = ca_public_key["p"]
19        q = ca_public_key["q"]
20        g = ca_public_key["g"]
21
22    # Load the CSR file
23    with open(os.path.join(SHARED_DIR_CSR, "csr.json"), "r") as f:
24        csr = json.load(f)
25
26    # Extract the public key from the CSR
27    csr_public_key_pem = csr["public_key"] # Full PEM string
28    hash_obj = hashlib.sha256(csr_public_key_pem.encode()).digest()
29    # Hash the PEM string directly
30    hash_value = int.from_bytes(hash_obj, byteorder='big') % q # Mod q
31
32    # Generate the DSA signature
33    k = random.randint(1, q - 1)
34    k_inv = pow(k, -1, q) # Modular inverse of k mod q
35    r = pow(g, k, p) % q
36    s = (k_inv * (hash_value + x * r)) % q
37    # Create the certificate JSON
38    cert = {
39        "certificate_id": "cert_001",
40        "issued_to": csr["common_name"],
41        "organization": csr["organization"],
42        "issued_by": "Jocker",
43        "public_key": csr["public_key"],
44        "signature": {"r": r, "s": s},
45        "validity_period": f"{datetime.now().date()} to {(datetime.now() + timedelta(days=365)).date()}",
46    }
47
48    # Save the certificate to the shared directory
49    os.makedirs(SHARED_DIR_CERT, exist_ok=True)
50    with open(os.path.join(SHARED_DIR_CERT, "cert.json"), "w") as f:
51        json.dump(cert, f, indent=4)
52
53    print("CA: Certificate signed and saved as 'cert.json'.")
54

```

بخش پنجم) ارتباط با مشتری با سرور و رد و بدل کردن پیام رمزی: توی این بخش اول مشتری به کانکشن با سرور ایجاد می‌کنه. بعد سرور ازش درخواست سرتیفیکیت می‌کنه. توی مرحله‌ی بعد مشتری سرتیفیکیت رو می‌فرسته برای سرور. سرور این درخواست رو می‌فرسته برای سی‌ای سرور تا تایید کنه که آیا امضا موثق یا نه. اگر موثق نبود که هیچی سرور یه پیام ارور می‌فرسته برای مشتری و برنامه تموم می‌شه. ولی اگر موثق باشه سرور یه کلید متقارن درست می‌کنه و اون کلید رو با استفاده از کلید عمومی مشتری رمز می‌کنه و بعد می‌فرسته برای مشتری. بعد مشتری کلید رو می‌گیره و رمزگشایی می‌کنه و یه پیام (همون پیامی که توی متن پروژه هست) رو با استفاده از کلید متقارن خودش رمز می‌کنه و می‌فرسته برای سرور. بعد سرور پیام رو دریافت می‌کنه و رمزگشایی می‌کنه و نمایش می‌ده. کل فرآیندی که باید رخ بده اینه. من کد این بخش‌ها رو این پایین قرار می‌دم:

```

1 def start_client():
2     """Client communicates with the server."""
3     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as client_socket:
4         client_socket.connect(("localhost", 8080))
5         print("Client: Connected to the server.")
6
7     # Server requests certification
8     request = client_socket.recv(1024).decode()
9     if request == "SEND_CERTIFICATE":
10        print("Client: Received certificate request from server.")
11
12        # Send CERT.json to the server
13        with open(os.path.join(SHARED_DIR_CERT, "cert.json"), "r") as f:
14            certificate = f.read()
15            client_socket.sendall(certificate.encode())
16            print("Client: Sent certificate to server.")
17
18        # Receive symmetric key and mode (encrypted)
19        response = client_socket.recv(4096).decode()
20        if response == "INVALID":
21            print("Client: Received INVALID certificate response. Closing connection.")
22        else:
23            encrypted_key, mode = json.loads(response)
24            print("Client: Received encrypted symmetric key and mode.")
25
26            # Decode and display the private key
27            with open(os.path.join(CLIENT_DIR, "client_private_key.pem"), "r") as f:
28                lines = f.read().splitlines()
29                base64_content = lines[1]
30                private_key = json.loads(base64.b64decode(base64_content))
31
32                d = private_key["d"]
33                n = private_key["n"]
34                p = private_key["p"]
35                q = private_key["q"]
36                e = private_key["e"]
37
38            # Decode and display the public key
39            with open(os.path.join(CLIENT_DIR, "client_public_key.pem"), "r") as f:
40                lines = f.read().splitlines()
41                base64_content = lines[1]
42                public_key = json.loads(base64.b64decode(base64_content))
43
44                n_pub = public_key["n"]
45                e_pub = public_key["e"]
46
47            # Decrypt the symmetric key with the private key
48            symmetric_key = pow(encrypted_key, d, n)
49            print(f"this is the symmetric key: {int_to_binary_array(symmetric_key)}")
50            print("
51            *****
52            ")
53            # Encrypt a message using CBC mode and send to server
54            iv = ["10101000"] * 16
55            message = "
56            There is nothing more to be said or to be done tonight, so hand me over my violin and let
57            us try to forget for half an hour the miserable weather and the still more miserable ways
58            of our fellowmen.
59            "
60            encrypted_message = EBC_mode_encryption(message, int_to_binary_array(
61            symmetric_key))
62            print(f"this is the encrypted message in client: {
63            decode_binary_array_with_errors(encrypted_message)}")
64            print("
65            *****
66            ")
67            client_socket.sendall(json.dumps(encrypted_message).encode())
68            print("Client: Sent encrypted message to server.")
69            print("
70            *****
71            ")

```

```

1 def start_server():
2     """Server communicates with the client and CA."""
3     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
4         server_socket.bind(("localhost", 8080))
5         server_socket.listen(1)
6         print("Server: Waiting for a client connection...")
7
8     conn, addr = server_socket.accept()
9     with conn:
10        print(f"Server: Connected to client at {addr}.")
11        conn.sendall("SEND_CERTIFICATE".encode())
12
13        # Receive CERT.json from client
14        certificate = conn.recv(4096).decode()
15        print("Server: Received certificate from client.")
16        # Validate certificate with CA
17        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ca_socket:
18            ca_socket.connect(("localhost", 9090))
19            ca_socket.sendall(certificate.encode())
20            print("Server: Sent certificate to CA.")
21
22            validation_result = ca_socket.recv(1024).decode()
23
24        if validation_result == "VALID":
25            print("Server: Certificate is valid.")
26
27            # Generate symmetric key and encrypt with client's public RSA key
28            symmetric_key = random.randint(1, 2**128)
29            print(f"this is the symmetric_key: {symmetric_key}")
30            print("
31            *****
32            ")
33            with open(os.path.join(CLIENT_DIR, "client_public_key.pem"), "r") as f:
34                lines = f.read().splitlines() # Split into Lines
35                base64_content = lines[1] # The second line
36                public_key = json.loads(base64.b64decode(base64_content))
37
38                e = public_key["e"]
39                n = public_key["n"]
40
41                encrypted_key = pow(symmetric_key, e, n)
42
43                conn.sendall(json.dumps([encrypted_key, "CBC"]).encode())
44                print("Server: Sent encrypted symmetric key to client.")
45
46                # Receive encrypted message and decrypt
47                data = b''
48                while True:
49                    chunk = conn.recv(4096)
50                    if not chunk:
51                        break # No more data, client closed
52                    data += chunk
53                # print(f"this is encrypted message length in server : {data}")
54                encrypted_message = json.loads(data.decode())
55                print(f"this is encrypted message in server: {encrypted_message}")
56                print("
57                *****
58                ")
59                print(f"and this is len of that: {len(encrypted_message)}")
60                print("
61                *****
62                ")
63                iv = ["10101000"] * 16
64                print(f"this is symmetric key binary: {int_to_binary_array(symmetric_key)}")
65                print("
66                *****
67                ")
68                decrypted_message = EBC_mode_decryption(
69                    encrypted_message, int_to_binary_array(symmetric_key)
70                )
71                decrypted_message = decrypted_message[:len(decrypted_message)//2]
72                print("server: binary of decrypted_message: ", decrypted_message)
73                print("Server: Decrypted message:", decode_binary_array_with_errors(
74                    decrypted_message))
75                print("this is len of decrypted message: ", len(decrypted_message))
76                print("
77                *****
78                ")
79            else:
80                print("Server: Certificate is invalid.")
81                conn.sendall("INVALID".encode())

```



```

1 def start_ca():
2     """CA validates certificates."""
3     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as ca_socket:
4         ca_socket.bind(("localhost", 9090))
5         ca_socket.listen(1)
6         print("CA: Waiting for server connection...")
7
8         conn, addr = ca_socket.accept()
9         with conn:
10             print(f"CA: Connected to server at {addr}.")
11
12             # Receive the certificate from the server
13             certificate = conn.recv(4096).decode()
14             print("CA: Received certificate from server.")
15
16             # Load the CA's public key
17             with open(os.path.join(CA_DIR, "ca_public_key.pem"), "r") as f:
18                 pem_content = f.read().strip().splitlines()
19                 if len(pem_content) < 3 or not pem_content[0].startswith("
-----BEGIN DSA PUBLIC KEY-----"):
20                     raise ValueError("Invalid or malformed CA public key file.")
21                 public_key_pem = pem_content[1] # Extract the Base64 content
22                 ca_public_key = json.loads(base64.b64decode(public_key_pem))
23
24             # Decode and Load JSON
25
26             # Ensure the public key components are integers
27             y = int(ca_public_key["y"])
28             p = int(ca_public_key["p"])
29             q = int(ca_public_key["q"])
30             g = int(ca_public_key["g"])
31
32             # Parse the received certificate
33             cert = json.loads(certificate)
34
35             # Extract the signature and validate it
36             r, s = cert["signature"]["r"], cert["signature"]["s"]
37
38             # Decode the public key in the certificate
39             cert_public_key_pem = cert["public_key"] # Full PEM string
40             hash_obj = hashlib.sha256(cert_public_key_pem.encode()).digest()
41             # Hash the PEM string directly
42             hash_value = int.from_bytes(hash_obj, byteorder='big') % q # Mod q
43
44             # Validate the signature
45             w = pow(s, -1, q)
46             u1 = (hash_value * w) % q
47             u2 = (r * w) % q
48             v = ((pow(g, u1) % p * pow(y, u2) % p) % p)
49             v = v % q
50
51             if v == r:
52                 print("CA: Certificate is valid.")
53                 conn.sendall("VALID".encode())
54             else:
55                 print("CA: Certificate is invalid.")
56                 conn.sendall("INVALID".encode())

```

بخش ششم) اجرا و خروجی برنامه: برای اینکه بتوانیم بخش مشتری و سرور رو انجام بدیم باید اول سرتیفیکیت و درخواستشو اینا رو کلا انجام بدیم. پس اول اون‌ها رو به صورت آفلاین انجام می‌دیم و بعد سوکت باز می‌کنیم با سرور و کارهای سروریمون رو انجام می‌دیم. کد این بخش هم این براتون قرار می‌دم ولی واقعا هیچ نکته‌ی خاصی نداره صرفا خوندن کده و هیچی هیچی نداره که نیاز به توضیح باشه:

```
1 # Ensure all directories exist
2 def setup_directories():
3     os.makedirs("shared_csr", exist_ok=True)
4     os.makedirs("shared_cert", exist_ok=True)
5     os.makedirs("client", exist_ok=True)
6     os.makedirs("ca", exist_ok=True)
7
```

```
1 def offline_setup():
2     print("--- Offline Setup ---")
3
4     # Generate RSA key pair for the client
5     private_key, public_key = generate_rsa_key_pair()
6     print("Client: RSA key pair generated.")
7
8
9     # Create a CSR and save it in the shared directory
10    create_csr()
11    print("Client: CSR created and saved to shared directory.")
12
13    # Generate DSA key pair for the CA
14    ca_private_key, ca_public_key = generate_dsa_key_pair()
15    print("CA: DSA key pair generated.")
16
17    # CA signs the CSR and generates a certificate
18    sign_csr_and_generate_cert()
19    print("CA: Certificate signed and saved to shared directory.")
20
```

```
1  # Online steps (testing client, server, and CA communication)
2  def online_test():
3      print("\n--- Online Test ---")
4
5      # Start CA server
6      def run_ca():
7          start_ca()
8
9      ca_thread = threading.Thread(target=run_ca, daemon=True)
10     ca_thread.start()
11
12     # Start server
13     def run_server():
14         start_server()
15
16     server_thread = threading.Thread(target=run_server, daemon=True
)
17     server_thread.start()
18
19     # Allow CA and server to initialize
20     time.sleep(2)
21
22     # Start client
23     def run_client():
24         start_client()
25
26     client_thread = threading.Thread(target=run_client, daemon=True
)
27     client_thread.start()
28
29     # Wait for threads to complete
30     client_thread.join()
31     server_thread.join()
32     ca_thread.join()
33
```

```
1 if __name__ == "__main__":
2     # Step 1: Setup directories
3     setup_directories()
4
5     # Step 2: Run offline setup
6     offline_setup()
7
8     # Step 3: Test online communication
9     online_test()
```

--- Offline Setup ---

Generating RSA Key Pair...

RSA Key Pair Generated and Saved in PEM Format.

Client: RSA key pair generated.

Client: Creating CSR...

Client: CSR created and saved to shared directory.

Client: CSR created and saved to shared directory.

Generating DSA Key Pair...

DSA Key Pair Generated and Saved in PEM Format.

CA: DSA key pair generated.

CA: Signing CSR and generating certificate...

CA: Certificate signed and saved as 'cert.json'.

CA: Certificate signed and saved to shared directory.

--- Online Test ---

Server: Waiting for a client connection...

CA: Waiting for server connection...

Client: Connected to the server.

Server: Connected to client at ('127.0.0.1', 12830).

Client: Received certificate request from server.

Client: Sent certificate to server.

Server: Received certificate from client.

Server: Sent certificate to CA.

CA: Connected to server at ('127.0.0.1', 12831).

CA: Received certificate from server.

CA: Certificate is valid.

Server: Certificate is valid.

this in the symmetric_key: 136081091274144270921747459055515166441

```

Server: Sent encrypted symmetric key to client.
Client: Received encrypted symmetric key and mode.
this is the symmetric key: ['11001100', '11000000', '10000111', '10011011', '00010010', '01001110', '01000001', '00101101', '01001001', '11011010', '1111100
1', '10111001', '01000111', '01001010', '11001101', '1101001']
*****
this is the encrypted message in client: nt0C0Thk00 b00000 mpA00e 00 q 070so?-0onL000p0my0/020Bme*/0
00n 'x0i00etk00'0P t0000.ou
DK0e \00[h
mi0HLe 07000r úP07er00smoR00000urw<00[ay0e0000R0000wm00e000
*****
Client: Sent encrypted message to server.
*****
this is encrypted message in server: ['00100000', '01101110', '01110100', '10010001', '00011000', '01000011', '11010001', '00000000', '01010100', '01101000',
'11010000', '10111010', '10001010', '00001111', '11000110', '00010100', '00100000', '01100010', '10111011', '10000101', '10100011', '11010000', '00010000',
'11000010', '00100000', '01101101', '00000010', '00110000', '01000001', '11110011', '10011011', '11100110', '01100101', '00100000', '11101111', '11101110',
'11001100', '10000101', '00011001', '11101110', '10000100', '00011001', '10101010', '01110011', '10000101', '01101110', '01001100', '10001101', '10110001', '10010111',
'01101111', '00111111', '00010101', '00101101', '11101110', '10011111', '10000101', '01101111', '01101110', '01001100', '10001101', '10110001', '10010111',
'01110000', '11011001', '01101101', '01111001', '10010110', '10111111', '00110010', '11011111', '00110010', '11011111', '01000010', '00011001', '10000100', '10011101',
'01101110', '00100000', '00100111', '11011001', '01111000', '01101011', '10100101', '01100101', '01101110', '01101100', '10100001', '00011001', '01101111', '01101101',
'00010101', '10001010', '10001010', '10010100', '00100100', '00101010', '10001100', '01100001', '01101100', '00100110', '00111011', '00001101', '01000100', '01001011',
'10001110', '01100101', '00100000', '01011100', '11110000', '10001110', '01011011', '01101000', '00001010', '01101101', '01101001', '00001111', '00000001',
'00010110', '11010111', '01001000', '01001100', '01100101', '00100000', '11010010', '00110111', '10000110', '11100110', '11000110', '11110001', '01110010',
'00100000', '11000011', '10111010', '01010000', '00011111', '11011101', '00110111', '01100101', '01111000', '00011001', '00011101', '11011111', '10111011', '01110011',
'00111100', '10111001', '10001010', '01011011', '01011101', '01100001', '01111001', '10000100', '01100101', '10011001', '11111110', '10110110', '10100001',
'00000000', '00000000', '01010010', '10110000', '00001111', '10111001', '11001110', '11010001', '01110111', '01101101', '10000010', '10111100', '01100101',
'10000111', '10110000', '11011110']
*****
and this is len of that: 208

```

```

this is symmetric key binary: ['11001100', '11000000', '10000111', '10011011', '00010010', '01001110', '01000001', '00101101', '01001001', '11011010', '1111
1001', '10111001', '01000111', '01001010', '11001101', '1101001']
*****
server: binary of decrypted message: ['01010100', '01101000', '01100101', '01110010', '01100101', '00100000', '01101001', '01110011', '00100000', '01101000', '01101000', '01101110',
'01101111', '00100000', '01100010', '01100101', '00100000', '01110011', '01100001', '01100100', '00100000', '01101111', '01110010', '00100000',
'01110100', '01101111', '00100000', '01100101', '00100000', '01100101', '00100000', '01100100', '00100000', '01110011', '01101111', '00100000', '01101000', '01100001', '01101110',
'01100100', '00100000', '01101101', '01100101', '00100000', '01101111', '01101110', '01100101', '01100100', '00100000', '01101101', '01111001', '01110010', '01100101',
'01101100', '01101001', '01101111', '01101100', '01101001', '01101110', '00100000', '01100001', '01101110', '01100100', '00100000', '01101100', '01100101',
'01110100', '00100000', '01110101', '01110011', '00100000', '01110100', '01110010', '01111001', '00100000', '01110100', '01101111', '00100000', '01100110',
'01101111', '01110010']
Server: Decrypted message: There is nothing more to be said or to be done tonight, so hand me over my violin and let us try to forg
this is len of decrypted message: 104
*****

```

این هم از گزارش فاز ۲ پروژه‌ی رمزنگاری. امیدوارم که خوشتون اومده باشه.